

**LAPORAN PROJECT UJIAN AKHIR SEMESTER
MATA KULIAH PENGOLAHAN SINYAL DIGITAL
TIM 01 2023 A**



**CLEANIFY PRO: INOVASI PENINGKATAN KUALITAS AUDIO DENGAN
TEKNOLOGI REDUKSI NOISE BERBASIS FINITE IMPULSE RESPONSE (FIR)
MENGUNAKAN LOW-PASS FILTER**

Ditulis Oleh:

Nova Aulia Agustin	NIM. 23031544022
Friza Chintia Putri	NIM. 23031544188
Rizka Ayu Rahmadhani	NIM. 23031544224

Dosen Pengampu:

Dr. Atik Wintarti, M.Kom.	NIP.196610121991032002
Kartika Chandra Dewi, S.Si., M.Si.	NIP.199312102024062002

**PROGRAM STUDI SARJANA SAINS DATA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS NEGERI SURABAYA
2024**

DAFTAR ISI

DAFTAR ISI.....	2
BAB I.....	4
PENDAHULUAN	4
1.1 Latar Belakang	4
1.2 Tujuan	5
1.3 Manfaat	5
1.4 Rumusan Masalah.....	5
BAB II	6
TINJAUAN PUSTAKA.....	6
2.1 Pengolahan Sinyal Digital.....	6
2.2 Teknik-Teknik Pengolahan Sinyal	6
2.3 Studi Terkait.....	7
BAB III.....	8
SOURCE CODE	8
1. Konversi File Audio MP3 Ke File Audio WAV	8
2. Menampilkan Audio Original.....	9
3. Membuat Audio Noise	10
4. Combine noise audio dengan original audio	11
5. Filtering Audio	12
6. Perbandingan Gelombang Sinyal Original Audio dan Hasil Filtering.....	13
7. Perbandingan Korelasi Original Audio dengan Audio Hasil Filtering	14
BAB IV	15
HASIL DAN PEMBAHASAN.....	15
1. Konversi File Audio MP3 Ke File Audio WAV	15
2. Menampilkan Informasi dari Audio File Original	15
3. Membuat Audio File Noise.....	16
4. Combine noise audio dengan original audio	18
5. Implementasi Filtering untuk mereduksi Noise.....	19

6. Perbandingan Gelombang Sinyal Original Audio dan Hasil Filtering.....	21
7. Perbandingan Korelasi Original Audio dengan Audio Hasil Filtering	22
8. Graphic User Interface (GUI) Cleanify Pro	23
BAB V	31
KESIMPULAN DAN SARAN	31
5.1 Kesimpulan	31
5.2 Saran.....	31
DAFTAR PUSTAKA.....	32

BAB I

PENDAHULUAN

1.1 Latar Belakang

Peningkatan kualitas suara atau audio menjadi sangat penting dalam berbagai aplikasi, terutama ketika kualitas suara dapat mempengaruhi kenyamanan dan efektivitas komunikasi. Dalam rekaman audio atau komunikasi digital, noise yang tidak diinginkan sering kali mengganggu kualitas suara, yang dapat menyebabkan ketidakjelasan dan gangguan. Hal ini menjadi tantangan besar dalam berbagai sektor, seperti rekaman musik, layanan telekomunikasi, sistem pengenalan suara, dan perangkat audio lainnya. Noise dapat muncul akibat gangguan sinyal pada saat perekaman atau transmisi, atau karena keterbatasan perangkat keras yang digunakan dalam perekaman. Sebagai contoh, pada komunikasi telepon atau video call, noise dapat mengurangi kualitas percakapan, yang mengarah pada ketidakjelasan atau kesulitan dalam memahami pesan. Begitu pula dalam industri musik dan film, noise yang muncul dalam proses perekaman dapat merusak kualitas keseluruhan karya tersebut, sehingga membutuhkan teknologi pengolahan sinyal yang canggih untuk memperbaikinya.

Pesatnya perkembangan teknologi komunikasi, kualitas suara menjadi salah satu faktor penting dalam penyampaian informasi. Noise atau kebisingan dalam sinyal audio dapat mengganggu pemahaman dan pengalaman pendengar, sehingga perlu adanya sistem pengurangan noise yang efektif. Noise sering kali muncul akibat gangguan pada sinyal yang mengurangi kejernihan suara dan mengganggu komunikasi. Dalam konteks ini, penerapan teknologi pengolahan sinyal digital menjadi sangat krusial untuk meningkatkan kualitas audio. Salah satu metode yang banyak digunakan untuk mengatasi masalah noise adalah penerapan filter, khususnya low-pass filter. Low-pass filter untuk frekuensi rendah melewati frekuensi tinggi yang biasanya merupakan noise dalam sinyal audio. Dalam penelitian ini, Salih (2017) menjelaskan bahwa filter adalah jaringan yang memproses sinyal secara bergantung pada frekuensi, di mana filter tipe Finite Impulse Response (FIR) dikenal memiliki keunggulan dalam stabilitas dan fase linier. Filter FIR, yang bersifat non-reaktif terhadap perubahan frekuensi, menawarkan keuntungan dalam pengolahan sinyal audio karena dapat memberikan respon yang lebih tepat dan tidak memperkenalkan distorsi fase. Dengan menggunakan low-pass filter berbasis FIR, noise dalam sinyal audio dapat diminimalisir, menghasilkan suara yang lebih jernih dan berkualitas tinggi.

Penggunaan teknologi FIR dalam sistem pengurangan noise menawarkan solusi yang efisien dan efektif untuk meningkatkan kualitas audio. Oleh karena itu, penelitian ini bertujuan untuk mengembangkan inovasi bernama "CLEANIFY PRO", yang memanfaatkan teknologi reduksi noise berbasis FIR untuk meningkatkan kualitas audio. Dengan pendekatan yang

sistematis, CLEANIFY PRO diharapkan dapat memberikan pengalaman audio yang lebih baik bagi pengguna, dengan fokus pada pengurangan noise tanpa mengorbankan kualitas sinyal asli.

1.2 Tujuan

Tujuan dari penelitian ini adalah untuk mengembangkan sistem pengolahan sinyal digital yang dapat meningkatkan kualitas audio dengan cara mengurangi noise yang tidak diinginkan.

1. Meningkatkan kualitas audio melalui pengolahan sinyal digital, sehingga menghasilkan audio yang lebih jernih dan jelas.
2. Membandingkan sinyal audio sebelum dan setelah noise ditambahkan yang sudah di filtering.
3. Evaluasi kualitas audio yang dihasilkan setelah pengolahan sinyal dengan menggunakan pengukuran korelasi dan Mean Squared Error (MSE)

1.3 Manfaat

1. Meningkatkan pemahaman dan pengetahuan dalam bidang pengolahan sinyal digital, khususnya pada penerapan teknologi FIR dan low-pass filter untuk pengurangan noise dalam audio.
2. Memberikan kontribusi pada perkembangan teknologi pengolahan sinyal audio yang dapat digunakan untuk penelitian lebih lanjut dalam meningkatkan kualitas suara.
3. Memungkinkan pengembangan perangkat lunak atau perangkat keras yang dapat meningkatkan pengalaman pengguna dalam mendengarkan audio yang lebih bersih dan jernih.

1.4 Rumusan Masalah

1. Bagaimana cara mengurangi noise dalam sinyal audio menggunakan teknologi pengolahan sinyal digital berbasis Finite Impulse Response (FIR)?
2. Bagaimana perbandingan sinyal audio sebelum dan setelah noise ditambahkan yang sudah di filtering.
3. Bagaimana Evaluasi kualitas audio yang dihasilkan setelah pengolahan sinyal dengan menggunakan pengukuran korelasi dan Mean Squared Error (MSE)

BAB II

TINJAUAN PUSTAKA

2.1 Pengolahan Sinyal Digital

Pengolahan sinyal digital (*Digital Signal Processing* - DSP) adalah teknik yang digunakan untuk memanipulasi sinyal yang telah dikonversi menjadi format digital. Sinyal ini biasanya berupa urutan angka yang mewakili sampel dari variabel kontinu dalam domain waktu, ruang, atau frekuensi. *Digital Signal Processing* - DSP memungkinkan berbagai operasi seperti pengurangan noise, kompresi data, dan analisis spektral.

Pengolahan Sinyal Digital (*Digital Signal Processing* - DSP) merupakan suatu teknik untuk menganalisis, memanipulasi, dan memodifikasi sinyal dalam bentuk digital dengan tujuan meningkatkan kualitas sinyal atau mengekstraksi informasi penting dari sinyal tersebut. Sinyal digital sendiri diperoleh dari sinyal analog melalui proses *sampling* dan *quantization*, di mana sinyal kontinu diubah menjadi data diskrit yang dapat diolah oleh perangkat digital. Proses utama dalam pengolahan sinyal digital meliputi *sampling*, yaitu pengambilan sampel sinyal analog pada interval waktu tertentu, dan *quantization*, yang mengonversi sampel menjadi nilai diskrit atau bilangan biner. Selanjutnya, sinyal dapat diproses dengan metode *filtering* untuk memisahkan atau menghilangkan komponen frekuensi tertentu, serta menggunakan transformasi seperti *Fourier Transform* atau *Discrete Fourier Transform (DFT)* untuk menganalisis sinyal dalam domain frekuensi.

Pengolahan sinyal digital memiliki berbagai aplikasi, antara lain dalam audio *processing* untuk meningkatkan kualitas suara dan menghilangkan noise, komunikasi untuk kompresi dan transmisi data, pengolahan citra untuk memperbaiki kualitas gambar, serta dalam bidang medis untuk analisis sinyal seperti EKG dan MRI. Keunggulan DSP meliputi akurasi yang tinggi, fleksibilitas dalam implementasi melalui perangkat lunak, serta ketahanannya terhadap gangguan noise. Dengan kemampuannya dalam meningkatkan efisiensi dan kualitas sinyal, pengolahan sinyal digital memainkan peran penting dalam berbagai inovasi teknologi modern, termasuk sistem komunikasi digital, pemrosesan audio-visual, dan sistem kendali otomatis.

2.2 Teknik-Teknik Pengolahan Sinyal

Beberapa teknik utama dalam pengolahan sinyal digital meliputi:

1. Filter FIR: Filter ini memiliki koefisien tetap dan respons impuls terbatas. FIR umumnya lebih stabil dan mudah dirancang dibandingkan IIR.
2. Filter IIR: Filter ini menggunakan umpan balik, sehingga dapat menghasilkan respons impuls yang tidak terbatas. IIR lebih efisien dalam penggunaan memori dan komputasi tetapi bisa menjadi tidak stabil jika tidak dirancang dengan benar. Filter IIR dari (*Infinite Impulse Response*) adalah salah satu tipe dari filter digital yang dipakai pada aplikasi Digital Signal Processing (DSP). Keuntungan filter IIR antara lain adalah membutuhkan

koefisien yang lebih sedikit untuk respon frekuensi yang curam sehingga dapat mengurangi jumlah waktu komputasi.

3. *Discrete Fourier Transform* (DFT): DFT mengubah sinyal dari domain waktu ke domain frekuensi, memungkinkan analisis spektral.
4. *Fast Fourier Transform* (FFT): Merupakan algoritma efisien untuk menghitung DFT, mengurangi waktu komputasi secara signifikan.
5. Transformasi z: Berguna untuk analisis sistem diskrit dan desain filter digital, memberikan wawasan tentang stabilitas sistem.

2.3 Studi Terkait

Dalam lima tahun terakhir, banyak penelitian telah dilakukan terkait pengolahan sinyal digital. Beberapa di antaranya mencakup:

1. Penerapan Filter Adaptif: Penelitian mengenai penggunaan filter adaptif dalam pengurangan noise pada sinyal EEG menunjukkan efektivitasnya dalam meningkatkan kualitas data medis.
2. Kombinasi Teknik DSP dengan AI: Penelitian terbaru mengeksplorasi integrasi algoritma pembelajaran mesin dengan teknik DSP untuk meningkatkan akurasi dalam pengenalan suara dan citra.
3. Pengembangan Algoritma FFT: Penelitian di bidang algoritma FFT berfokus pada optimisasi untuk aplikasi real-time dalam telekomunikasi dan pemrosesan video.
4. Aplikasi di Biomedis: Banyak studi yang meneliti penggunaan DSP dalam analisis sinyal biomedis, seperti ECG dan EEG, untuk diagnosis penyakit serta monitoring kesehatan secara real-time.

Penelitian-penelitian ini menunjukkan perkembangan signifikan dalam teknik pengolahan sinyal digital serta aplikasinya di berbagai bidang teknologi modern.

BAB III

SOURCE CODE

1. Konversi File Audio MP3 Ke File Audio WAV

```
[1] !pip install pydub
Show hidden output

[2] from pydub import AudioSegment

def convert_mp3_to_wav(input_file, output_file):
    audio = AudioSegment.from_file(input_file, format="mp3")
    audio.export(output_file, format="wav")
    print(f"File berhasil dikonversi: {output_file}")

input_mp3 = "sada.mp3"
output_wav = "sada.wav"

convert_mp3_to_wav(input_mp3, output_wav)
Show hidden output
```

Kode di atas digunakan untuk mengonversi file audio berformat MP3 menjadi WAV menggunakan library *Pydub* di *Python*. Pertama, program mengimpor kelas *AudioSegment* dari *Pydub*, yang memungkinkan manipulasi file audio, seperti membuka, mengedit, dan mengonversi format audio. Fungsi utama dalam kode ini adalah *convert_mp3_to_wav*, yang memiliki dua parameter: *input_file* (file MP3 yang akan dikonversi) dan *output_file* (file WAV yang akan dihasilkan). Di dalam fungsi tersebut, *AudioSegment.from_file()* digunakan untuk membuka file MP3 yang diberikan sebagai input.

Metode ini secara otomatis mendeteksi format file yang akan dibaca, dan dalam hal ini, kita secara eksplisit menyebutkan format "mp3". Setelah audio berhasil dimuat, *audio.export()* digunakan untuk mengekspor file audio ke format WAV. Fungsi ini menerima dua parameter utama: nama file output dan format yang diinginkan, yaitu "wav". Setelah proses konversi selesai, fungsi menampilkan pesan bahwa file berhasil dikonversi ke format WAV menggunakan fungsi *print()*. Pada bagian akhir kode, fungsi *convert_mp3_to_wav* dipanggil dengan file input bernama sada.mp3 dan menghasilkan file output bernama sada.wav. Dengan demikian, kode ini menyederhanakan proses konversi format file audio yang umum digunakan dalam berbagai aplikasi pemrosesan dan analisis audio.

2. Menampilkan Audio Original

```
import librosa
import librosa.display
import matplotlib.pyplot as plt
import numpy as np
from IPython.display import Audio

original_file = "sada.wav"
audio, sr = librosa.load(original_file, sr=None)
duration = librosa.get_duration(y=audio, sr=sr)

print(f"Sample Rate: {sr} Hz")
print(f"Durasi: {duration:.2f} detik")

plt.figure(figsize=(10, 4))
librosa.display.waveshow(audio, sr=sr, color='blue', alpha=0.8)
plt.title("Gelombang Sinyal Original Audio")
plt.xlabel("Waktu (detik)")
plt.ylabel("Amplitudo")
plt.grid()
plt.show()

n = len(audio)
fft_audio = np.fft.fft(audio)
fft_freqs = np.fft.fftfreq(n, d=1/sr)

positive_freqs = fft_freqs[n//2:]
positive_fft = np.abs(fft_audio[n//2:])

dominant_freq = positive_freqs[np.argmax(positive_fft)]
print(f"Frekuensi Dominan: {dominant_freq:.2f} Hz")

plt.figure(figsize=(10, 6))
plt.plot(positive_freqs, positive_fft)
plt.title("Spektrum Frekuensi Audio")
plt.xlabel("Frekuensi (Hz)")
plt.ylabel("Magnitudo")
plt.grid()
plt.show()

print('Original Audio:')
Audio(original_file)
```

Kode di atas digunakan untuk menganalisis file audio dengan memanfaatkan library *Librosa*, *Matplotlib*, dan *NumPy*. Pertama, file audio "sada.wav" dimuat menggunakan fungsi *librosa.load()*, yang juga mengembalikan sample rate (sr) dan data audio itu sendiri. Durasi file audio dihitung dengan *librosa.get_duration()* dan informasi dasar seperti sample rate dan durasi ditampilkan. Selanjutnya, bentuk gelombang sinyal audio divisualisasikan dengan *librosa.display.waveshow()*, yang menggambarkan perubahan amplitudo sinyal terhadap waktu. Kemudian, transformasi Fourier (FFT) dihitung dengan *np.fft.fft()*, yang mengubah sinyal dari domain waktu ke domain frekuensi.

Spektrum frekuensi positif diambil dan magnitudo frekuensi dihitung menggunakan *np.abs()* untuk mendapatkan intensitas pada berbagai frekuensi. Frekuensi dominan, yaitu frekuensi dengan magnitudo tertinggi, ditemukan menggunakan *np.argmax()* dan ditampilkan. Plot terakhir menunjukkan spektrum frekuensi audio, menggambarkan bagaimana distribusi energi tersebar pada frekuensi tertentu. Terakhir, kode memutar file audio yang dimuat, memungkinkan pengguna untuk mendengarkan audio yang sedang dianalisis. Analisis ini memberikan gambaran tentang karakteristik frekuensi dan komponen utama dalam file audio.

3. Membuat Audio Noise

```
import numpy as np
import scipy.signal as signal
import matplotlib.pyplot as plt
import soundfile as sf
from IPython.display import Audio, display

sr = 44100
duration = 7.45
base_freq = 189.84
additional_freq = 300
noise_bandwidth = 50

low_freq = additional_freq - noise_bandwidth / 2
high_freq = additional_freq + noise_bandwidth / 2

white_noise = np.random.normal(0, 1, int(sr * duration))

sos = signal.butter(4, [low_freq, high_freq], btype='bandpass', fs=sr, output='sos')
filtered_noise = signal.sosfilt(sos, white_noise)

output_filename = "sada_noise.wav"
sf.write(output_filename, filtered_noise, sr)
print(f"Noise tambahan telah disimpan ke file: {output_filename}")

display(Audio(data=filtered_noise, rate=sr))

plt.figure(figsize=(10, 4))
plt.plot(np.linspace(0, duration, len(filtered_noise)), filtered_noise, color='blue', alpha=0.8)
plt.title("Gelombang Noise di Frekuensi Lebih Tinggi")
plt.xlabel("waktu (detik)")
plt.ylabel("amplitudo")
plt.grid()
plt.show()

frequencies, power_density = signal.periodogram(filtered_noise, sr)

plt.figure(figsize=(10, 4))
plt.plot(frequencies, power_density, color='green', alpha=0.8)
plt.title("Frekuensi Noise (Periodogram)")
plt.xlabel("Frekuensi (Hz)")
plt.ylabel("Densitas Daya")
plt.xlim(0, 1000)
plt.grid()
plt.show()
```

Kode di atas dirancang untuk menghasilkan dan menganalisis noise audio dengan rentang frekuensi tertentu. Pertama, parameter dasar ditentukan, seperti sample rate (sr) sebesar 44100 Hz, durasi noise (duration) 7.45 detik, frekuensi dasar (base_freq) 189.84 Hz, frekuensi tambahan (additional_freq) 300 Hz, dan bandwidth noise (noise_bandwidth) 50 Hz. Rentang frekuensi noise dihitung sebagai batas bawah (low_freq) dan batas atas (high_freq) berdasarkan nilai frekuensi tambahan dan bandwidth.

Selanjutnya, white noise dibuat menggunakan fungsi `np.random.normal()`, yang menghasilkan distribusi Gaussian dengan mean nol. Noise ini kemudian difilter menggunakan filter band-pass digital yang dirancang dengan fungsi `signal.butter()`. Filter ini dibuat dengan orde 4 dan diimplementasikan dalam bentuk second-order sections (sos) untuk stabilitas numerik. Noise hasil filter dihitung dengan fungsi `signal.sosfilt()`, yang memastikan hanya noise dalam rentang frekuensi tertentu yang lolos. Noise yang dihasilkan disimpan dalam file WAV menggunakan `soundfile.write()` dan diberi nama "sada_noise.wav". Kode juga memungkinkan untuk memutar audio noise yang telah dibuat dengan menggunakan `IPython.display.Audio()`.

Gelombang noise divisualisasikan menggunakan Matplotlib dalam domain waktu, menampilkan amplitudo noise terhadap waktu. Untuk analisis lebih lanjut, spektrum frekuensi noise dihitung menggunakan fungsi `signal.periodogram()`, yang memberikan distribusi daya terhadap frekuensi. Grafik spektrum frekuensi menunjukkan distribusi energi noise, dibatasi hingga 1000 Hz untuk fokus pada frekuensi rendah. Visualisasi ini membantu memahami bagaimana energi noise tersebar di rentang frekuensi tertentu. Kode ini secara keseluruhan memanfaatkan kombinasi metode simulasi, filtering, dan analisis spektral untuk mempelajari karakteristik noise.

4. Combine noise audio dengan original audio

```
import numpy as np
import librosa
import librosa.display
import matplotlib.pyplot as plt
import soundfile as sf
from IPython.display import Audio

def combine_audio_simple(file1, file2, output_file, weight1=0.5, weight2=0.5):
    audio1, sr1 = librosa.load(file1, sr=None)
    audio2, sr2 = librosa.load(file2, sr=None)

    min_len = min(len(audio1), len(audio2))
    audio1 = audio1[:min_len]
    audio2 = audio2[:min_len]

    combined_audio = weight1 * audio1 + weight2 * audio2
    combined_audio = combined_audio / np.max(np.abs(combined_audio))
    sf.write(output_file, combined_audio, sr1)

    return audio1, audio2, combined_audio, sr1

original_file = "sada.wav"
noise_file = "sada_noise.wav"
output_file = "sada_combined.wav"

audio1, audio2, combined_audio, sr = combine_audio_simple(original_file, noise_file, output_file, weight1=0.7, weight2=0.3)

plt.figure(figsize=(12, 6))
plt.plot(np.linspace(0, len(audio1)) / sr, len(audio1)), audio1, color='blue', alpha=0.6, label="Original")
plt.plot(np.linspace(0, len(audio2)) / sr, len(audio2)), audio2, color='red', alpha=0.6, label="Noise")
# plt.plot(np.linspace(0, len(combined_audio)) / sr, len(combined_audio)), combined_audio, color='green', alpha=0.6, label="Combined")

plt.title("Gelombang Sinyal: Original, Noise, dan Gabungan")
plt.xlabel("Waktu (detik)")
plt.ylabel("Amplitudo")
plt.legend()
plt.grid()
plt.show()

plt.figure(figsize=(12, 6))
plt.plot(np.linspace(0, len(combined_audio)) / sr, len(combined_audio)), combined_audio, color='purple', alpha=0.8, label="Combined")
plt.title("Gelombang Sinyal: Gabungan")
plt.xlabel("Waktu (detik)")
plt.ylabel("Amplitudo")
plt.legend()
plt.grid()
plt.show()

print('Combined Audio')
Audio(output_file)
```

Kode ini bertujuan untuk menggabungkan dua file audio, yaitu audio asli (sada.wav) dan noise (sada_noise.wav), dengan bobot tertentu, sehingga menghasilkan file audio gabungan (sada_combined.wav). Proses dimulai dengan membaca kedua file audio menggunakan `librosa.load()` untuk mendapatkan data audio beserta sample rate-nya. Panjang kedua audio disesuaikan dengan memotong audio yang lebih panjang berdasarkan panjang file terpendek. Kemudian, kedua audio digabung menggunakan bobot yang telah ditentukan (`weight1` untuk audio asli dan `weight2` untuk noise), dengan kombinasi linear: $\text{combined_audio} = \text{weight1} \times \text{audio1} + \text{weight2} \times \text{audio2}$. Hasil gabungan ini dinormalisasi agar amplitudo berada dalam rentang $[-1, 1]$, dan disimpan dalam file WAV menggunakan `soundfile.write()`. Selain itu, kode juga menampilkan visualisasi gelombang sinyal audio asli, noise, dan gabungannya menggunakan `matplotlib`, untuk memberikan gambaran tentang perubahan karakteristik sinyal. Terakhir, audio gabungan dapat langsung diputar menggunakan `IPython.display.Audio()` untuk memastikan hasil penggabungan sesuai. Kode ini sangat berguna untuk simulasi pengolahan sinyal audio dengan noise, serta untuk menguji efektivitas metode reduksi noise.

5. Filtering Audio

```
!pip install noisereducer
Show hidden output

import librosa
import noisereducer as nr
import soundfile as sf
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import firwin, lfilter
from IPython.display import Audio, display

def apply_fir_filter(audio, sr, cutoff=200):
    nyquist = sr / 2
    numtaps = 1
    fir_coeff = firwin(numtaps, cutoff / nyquist, pass_zero=True)
    filtered_audio = lfilter(fir_coeff, [1.0], audio)
    return filtered_audio

def enhance_audio_with_filter(audio, sr, noise_duration=1.0, cutoff=200):
    noise_clip = audio[int(noise_duration * sr)]
    audio_denixed = nr.reduce_noise(y=audio, sr=sr, y_noise=noise_clip)

    audio_filtered = apply_fir_filter(audio_denixed, sr, cutoff=cutoff)

    rms = np.sqrt(np.mean(audio_filtered**2))
    target_rms = rms * 3
    gain_factor = target_rms / rms
    audio_enhanced = audio_filtered * gain_factor

    return audio_enhanced

audio, sr = librosa.load('data_combined.wav', sr=None)
audio_enhanced = enhance_audio_with_filter(audio, sr, noise_duration=1.0, cutoff=200)
sf.write('data_enhanced.wav', audio_enhanced, sr)

plt.figure(figsize=(8, 4))
plt.plot(np.linspace(0, len(audio) / sr, len(audio)), audio, color='purple', alpha=0.5, label='Combined Audio')
plt.title('Sebelum Sinyal Original Audio dan Noise Audio')
plt.xlabel('Time (seconds)')
plt.ylabel('Amplitude')
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
display(Audio(data=audio, rate=sr))

plt.figure(figsize=(8, 4))
plt.plot(np.linspace(0, len(audio_enhanced) / sr, len(audio_enhanced)), audio_enhanced, color='rebeccapurple', alpha=0.5, label='Enhanced Audio (Noise Reduced & Filtered)')
plt.title('Enhanced Audio with Noise Reduction and Low-Pass Filter')
plt.xlabel('Time (seconds)')
plt.ylabel('Amplitude')
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
display(Audio(data=audio_enhanced, rate=sr))

plt.figure(figsize=(8, 4))
plt.plot(np.linspace(0, len(audio_enhanced) / sr, len(audio_enhanced)), audio_enhanced, color='green', alpha=1.0, label='Final Enhanced Audio')
plt.title('Final Enhanced Audio (Noise Reduced, Filtered & Louder)')
plt.xlabel('Time (seconds)')
plt.ylabel('Amplitude')
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
display(Audio(data=audio_enhanced, rate=sr))
```

Kode ini bertujuan untuk meningkatkan kualitas audio dengan tiga langkah utama, yaitu noise reduction, filtering, dan penyesuaian amplitudo. Pertama, audio yang berisi sinyal campuran (original + noise) dimuat menggunakan librosa. Kemudian, bagian awal audio selama durasi tertentu (default 1 detik) digunakan sebagai sampel noise, dan noise reduction diterapkan menggunakan library noisereducer untuk mengurangi komponen noise pada audio. Setelah noise berkurang, dilakukan proses filtering menggunakan FIR (Finite Impulse Response) low-pass filter dari scipy.signal.firwin dan lfilter. Filter ini memblokir frekuensi di atas batas tertentu (default 200 Hz), sehingga komponen frekuensi tinggi yang cenderung berupa noise dapat dihilangkan. Selanjutnya, amplitudo audio yang telah difilter diperkuat dengan menyesuaikan RMS (Root Mean Square) agar audio menjadi lebih nyaring.

Hasil akhir audio yang sudah diperbaiki kemudian disimpan menggunakan soundfile dan divisualisasikan dalam tiga grafik: audio gabungan asli, audio setelah noise reduction dan filtering, serta audio akhir dengan penyesuaian amplitudo. Setiap langkah dilengkapi dengan audio playback agar pengguna dapat mendengar perbedaannya secara langsung.

6. Perbandingan Gelombang Sinyal Original Audio dan Hasil Filtering

```
import librosa
import matplotlib.pyplot as plt
import numpy as np
from IPython.display import Audio, display

original_audio, sr_original = librosa.load('sada.wav', sr=None)
enhanced_audio, sr_enhanced = librosa.load('sada_hasil.wav', sr=None)

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(np.linspace(0, len(original_audio) / sr_original, len(original_audio)), original_audio, color='blue', alpha=0.6, label="Original Audio")
plt.title("Original Audio")
plt.xlabel("Time (seconds)")
plt.ylabel("Amplitude")
plt.legend()
plt.grid()

plt.subplot(1, 2, 2)
plt.plot(np.linspace(0, len(enhanced_audio) / sr_enhanced, len(enhanced_audio)), enhanced_audio, color='green', alpha=0.8, label="Enhanced Audio")
plt.title("Final Filtering Audio")
plt.xlabel("Time (seconds)")
plt.ylabel("Amplitude")
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()

print('Original Audio')
display(Audio(data=original_audio, rate=sr_original))
print('Filtering Audio')
display(Audio(data=enhanced_audio, rate=sr_enhanced))

plt.figure(figsize=(8, 4))
plt.plot(np.linspace(0, len(original_audio) / sr_original, len(original_audio)), original_audio, color='blue', alpha=0.6, label="Original Audio")
plt.plot(np.linspace(0, len(enhanced_audio) / sr_enhanced, len(enhanced_audio)), enhanced_audio, color='green', alpha=0.6, label="Filtering Audio")
plt.title("Original and Filtering Audio")
plt.xlabel("Time (seconds)")
plt.ylabel("Amplitude")
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```

Kode ini membandingkan audio original dan audio hasil filtering melalui visualisasi grafik dan playback audio. Visualisasi pertama menampilkan dua subplot: grafik kiri menunjukkan audio original berwarna biru, yang masih mengandung noise dengan amplitudo yang kacau, sedangkan grafik kanan menampilkan audio hasil filtering berwarna hijau, yang telah melalui proses noise reduction, low-pass filtering, dan amplifikasi, sehingga sinyal menjadi lebih bersih dan stabil. Selanjutnya, kedua audio diputar untuk memberikan perbandingan auditori, di mana audio original terdengar lebih berisik akibat noise, sedangkan audio hasil filtering terdengar lebih jernih dan nyaman didengar.

Grafik terakhir meng-overlay kedua sinyal dalam satu plot, dengan warna biru mewakili audio original dan warna hijau untuk audio hasil filtering. Perbedaan signifikan terlihat jelas, di mana sinyal hasil filtering memiliki noise yang berkurang drastis dan amplitudo yang lebih konsisten dibandingkan audio asli. Secara keseluruhan, output ini menunjukkan keberhasilan proses filtering dalam meningkatkan kualitas audio dengan mengurangi noise dan mempertajam sinyal utama.

7. Perbandingan Korelasi Original Audio dengan Audio Hasil Filtering

```
import librosa
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error

def resample_audio(audio, sr, target_sr):
    return librosa.resample(audio, orig_sr=sr, target_sr=target_sr)

def normalize_audio(audio):
    return audio / np.max(np.abs(audio))

def compare_audio_files(file1, file2):
    audio1, sr1 = librosa.load(file1, sr=None)
    audio2, sr2 = librosa.load(file2, sr=None)

    audio1 = normalize_audio(audio1)
    audio2 = normalize_audio(audio2)

    min_len = min(len(audio1), len(audio2))
    audio1 = audio1[:min_len]
    audio2 = audio2[:min_len]

    mse = mean_squared_error(audio1, audio2)

    correlation = np.corrcoef(audio1, audio2)[0, 1]

    plt.figure(figsize=(10, 5))
    plt.plot(audio1, label="Original Audio", alpha=0.7)
    plt.plot(audio2, label="Filtered Audio", alpha=0.7)
    plt.legend()
    plt.title("Waveform Comparison")
    plt.show()

    print(f"Audio1 Mean: {np.mean(audio1):.5f}, Std: {np.std(audio1):.5f}")
    print(f"Audio2 Mean: {np.mean(audio2):.5f}, Std: {np.std(audio2):.5f}")

    return mse, correlation

original_file = 'sada.wav'
filtered_file = 'sada_hasil.wav'

mse, correlation = compare_audio_files(original_file, filtered_file)

print(f"Mean Squared Error (MSE): {mse:.5f}")
print(f"Korelasi: {correlation:.5f}")
```

Kode ini digunakan untuk membandingkan dua file audio, yaitu audio asli dan audio hasil filtering, dengan tujuan untuk menilai seberapa efektif proses filtering dalam mengurangi noise dan mempertahankan kualitas sinyal. Pertama, kedua file audio dimuat dan dinormalisasi, yaitu amplitudo setiap sinyal disesuaikan agar berada dalam rentang -1 hingga 1. Setelah itu, panjang kedua audio disesuaikan untuk memastikan perhitungan statistik dilakukan pada segmen yang setara. Perbandingan dilakukan dengan menghitung Mean Squared Error (MSE), yang mengukur perbedaan antara kedua sinyal, dan koefisien korelasi, yang menunjukkan tingkat kesamaan bentuk gelombang antara audio asli dan hasil filtering.

Nilai MSE yang lebih rendah dan korelasi yang lebih tinggi menandakan bahwa hasil filtering sangat mirip dengan audio asli. Visualisasi gelombang audio juga disediakan untuk mempermudah analisis perbedaan antara kedua sinyal. Selain itu, nilai rata-rata dan standar deviasi untuk masing-masing audio dihitung dan ditampilkan. Hasil dari perhitungan ini memberikan wawasan kuantitatif tentang seberapa baik proses filtering dalam mempertahankan kualitas audio asli sambil mengurangi noise.

BAB IV

HASIL DAN PEMBAHASAN

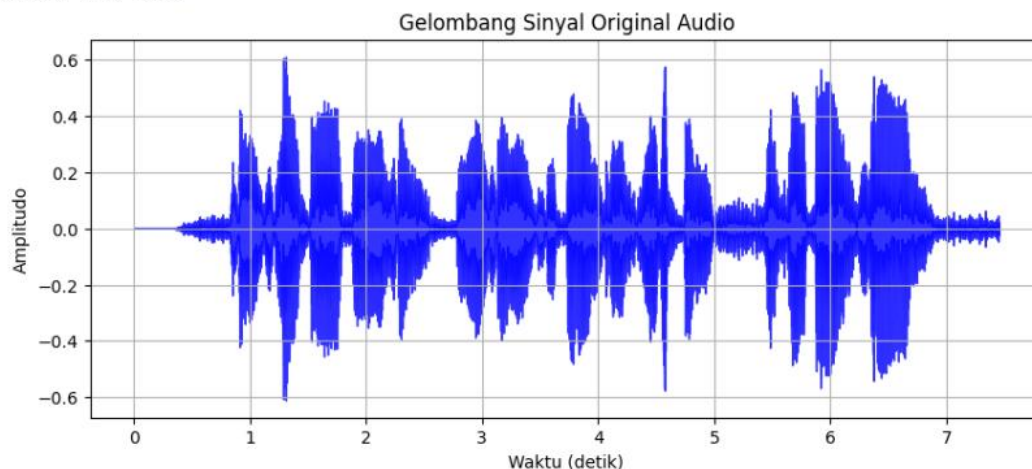
1. Konversi File Audio MP3 Ke File Audio WAV

Langkah awal dalam peningkatan kualitas audio dengan menerapkan teknologi reduksi noise berbasis Finite Impulse Response (FIR) menggunakan Low-Pass Filter adalah melakukan konversi format file audio. File audio asli tanpa noise, yang diberi nama *sada.mp3*, terlebih dahulu dikonversi ke format WAV. Konversi ini dilakukan karena format WAV memiliki keunggulan dibanding MP3 dalam hal kualitas audio. Format WAV lebih kompatibel untuk proses pengolahan sinyal digital, seperti penerapan filter, karena mempertahankan informasi amplitudo dan frekuensi secara lebih akurat.

2. Menampilkan Informasi dari Audio File Original

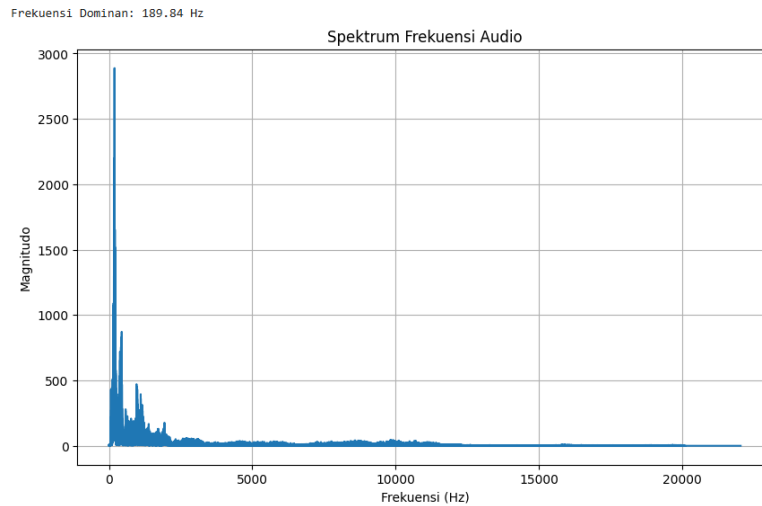
Berdasarkan analisis file audio *sada.wav* menggunakan pustaka Librosa, file audio memiliki beberapa karakteristik utama. Sample rate file audio adalah sebesar 44.100 Hz, yang menunjukkan bahwa data audio diambil 44.100 sampel per detik. Hal ini memastikan bahwa file memiliki kualitas yang memadai untuk kebutuhan pemrosesan audio, sesuai dengan standar kualitas CD audio, serta menampilkan informasi terkait Durasi file audio, yaitu 7,45 detik.

Sample Rate: 44100 Hz
Durasi: 7.45 detik



Selanjutnya, menampilkan visualisasi yang merupakan spektrum frekuensi yang dihasilkan dari transformasi Fourier, menunjukkan bahwa energi sinyal terkonsentrasi pada frekuensi dominan sebesar 189,84 Hz, dengan amplitudo yang jauh lebih kecil pada frekuensi lebih tinggi. Analisis ini mengindikasikan bahwa audio didominasi oleh

komponen frekuensi rendah, yang mungkin berasal dari elemen seperti suara vokal atau instrumen bass.



Dengan dilakukannya analisis informasi pada file original ini, dapat memberikan dasar penting untuk proses pengolahan sinyal selanjutnya, seperti pembuatan noise, combine noise dan original audio, serta pada penerapan filter *Low-Pass FIR* untuk mengurangi noise. Dengan mengetahui karakteristik ini, langkah-langkah berikutnya dapat dirancang untuk mempertahankan komponen frekuensi dominan sambil mengeliminasi noise.

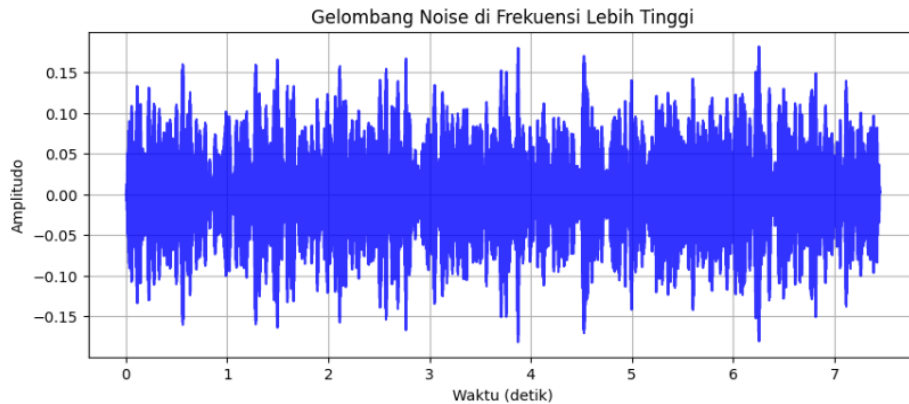
3. Membuat Audio File Noise

Setelah memperoleh informasi dari audio asli, langkah berikutnya adalah menghasilkan audio noise dengan rentang frekuensi tertentu. Pertama, parameter dasar ditetapkan, seperti sample rate (sr) sebesar 44100 Hz, durasi noise selama 7,45 detik, frekuensi dasar (base_freq) sebesar 189,84 Hz, frekuensi tambahan (additional_freq) 300 Hz, dan bandwidth noise (noise_bandwidth) sebesar 50 Hz. Rentang frekuensi noise kemudian dihitung dengan menentukan batas bawah (low_freq) dan batas atas (high_freq) berdasarkan frekuensi tambahan dan bandwidth yang telah ditentukan. Noise yang dihasilkan disimpan dalam file audio bernama `sada_noise.wav` dan diputar untuk verifikasi hasilnya.

Visualisasi gelombang sinyal noise yang ditambahkan untuk menunjukkan fluktuasi amplitudo yang sangat tinggi dalam rentang waktu 7,45 detik. Gelombang ini menunjukkan karakteristik noise dengan variasi amplitudo yang acak dan komponen frekuensi tinggi, dimana amplitudo yang berkisar antara -0.15 hingga 0.15, yang menandakan keberadaan noise berfrekuensi tinggi dengan perubahan amplitudo yang cepat terhadap waktu, sesuai dengan sifat dari noise yang dihasilkan melalui distribusi acak (white noise). Hal ini dengan tujuan untuk memperkenalkan gangguan frekuensi tinggi yang akan dihilangkan melalui penerapan filter *Low-Pass FIR* nantinya.

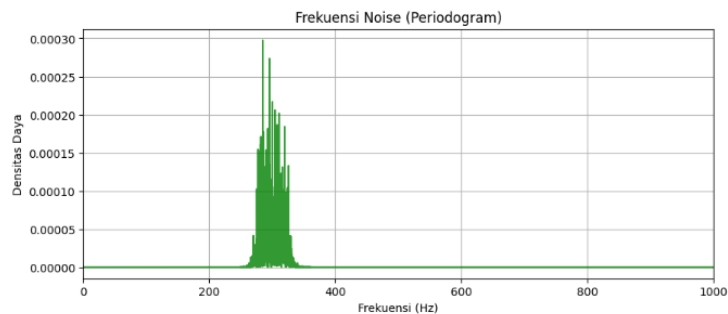
Noise tambahan telah disimpan ke file: sada_noise.wav

▶ 0:00 / 0:07 🔊 ⋮



Pada grafik selanjutnya menampilkan Periodogram, yang merepresentasikan distribusi daya noise dalam domain frekuensi. Sumbu horizontal menunjukkan frekuensi dalam satuan Hertz (Hz), sedangkan sumbu vertikal menunjukkan densitas daya atau Power Spectral Density (PSD). Pada analisis frekuensi noise (Periodogram), terdapat puncak yang signifikan sekitar 300 Hz, yang merupakan frekuensi utama dari noise yang telah ditambahkan. Dalam periodogram, densitas daya menunjukkan bagaimana energi dari noise tersebar pada berbagai frekuensi, dan di sini kita dapat melihat konsentrasi daya yang tinggi pada frekuensi noise yang diinginkan. Area di sekitar frekuensi 300 Hz menunjukkan adanya komponen noise yang dominan, sedangkan distribusi frekuensi lainnya menunjukkan komponen noise yang lebih tersebar atau lebih rendah intensitasnya.

Hasil ini memberikan gambaran mengenai keberadaan noise pada file audio dan frekuensi di mana noise tersebut lebih dominan. Langkah selanjutnya, yang berupa penerapan filter untuk mengurangi noise, akan difokuskan pada frekuensi yang terdeteksi, yaitu sekitar 300 Hz, dan diharapkan dapat meningkatkan kualitas audio dengan mengurangi komponen frekuensi yang tidak diinginkan.

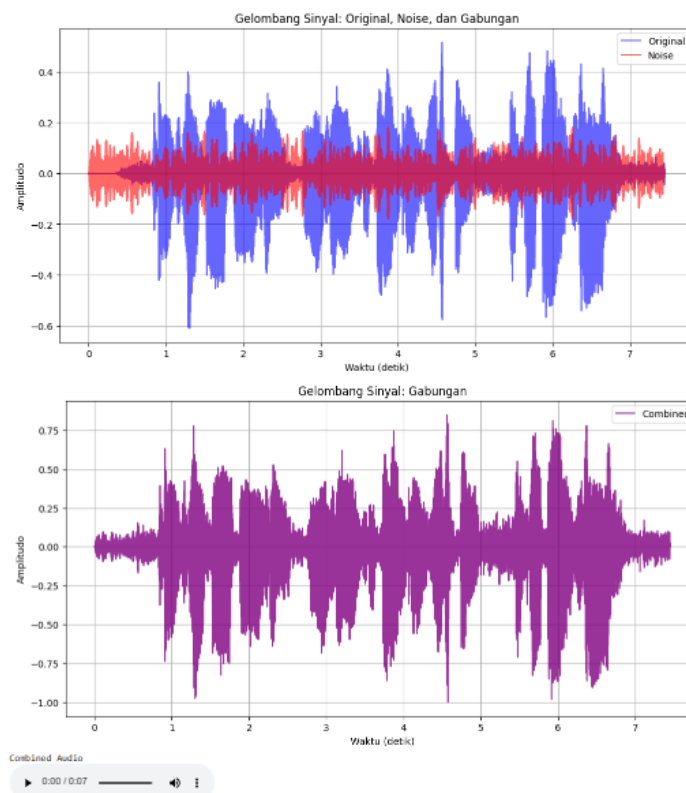


4. Combine noise audio dengan original audio

Setelah pembuatan noise dan konversi format audio pada file asli, langkah berikutnya adalah menggabungkan noise yang telah dibuat dengan audio asli. Proses penggabungan audio dilakukan dengan memberikan bobot pada kedua file audio (file asli dan noise). File audio asli diberi bobot 0.7 dan file noise diberi bobot 0.3, sehingga menghasilkan komposisi sinyal gabungan yang lebih dominan pada suara asli dengan sedikit tambahan noise. Setelah penggabungan, amplitudo sinyal gabungan diatur agar tidak melebihi batas yang dapat ditangani oleh format audio, memastikan bahwa sinyal tetap berada dalam rentang yang aman untuk pemutaran. Hasil gabungan ini dinormalisasi agar amplitudo berada dalam rentang $[-1,1]$, dan disimpan dalam file dengan format WAV.

Penggabungan ini bertujuan untuk mempelajari bagaimana noise dapat mempengaruhi kualitas audio dan bagaimana pemfilteran serta pengolahan sinyal selanjutnya dapat memperbaiki kualitas audio yang telah terkena oleh noise. Proses ini menjadi bagian penting dalam evaluasi keberhasilan teknik reduksi noise dalam aplikasi audio.

Selain itu, ditampilkannya visualisasi penggabungan gelombang sinyal audio original dengan audio noise yang telah dibuat sebelumnya.



Output pada gambar ini menunjukkan proses penggabungan antara sinyal audio original dan noise yang diilustrasikan dalam dua grafik. Pada grafik pertama (bagian atas), terdapat tiga komponen yang divisualisasikan, yaitu sinyal original berwarna biru, noise

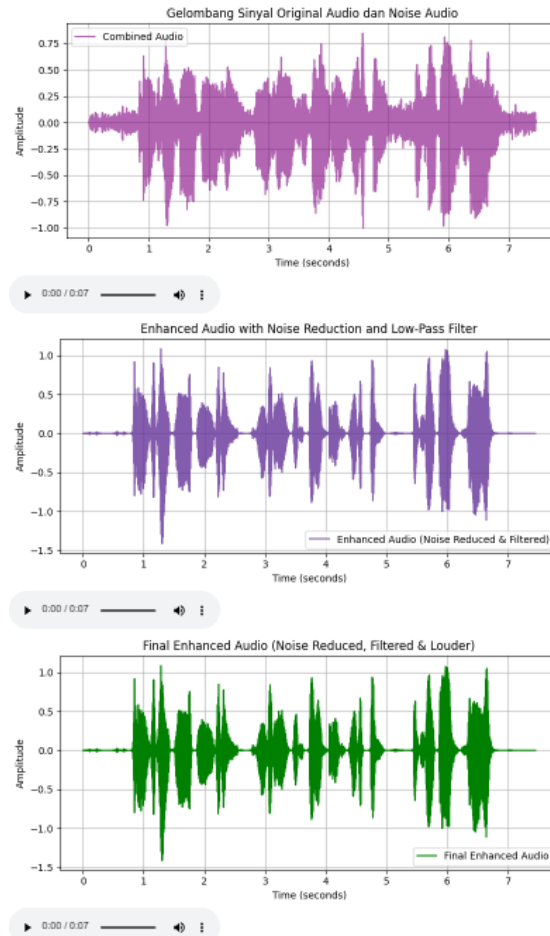
berwarna merah, dan gabungan keduanya. Sumbu horizontal menunjukkan waktu (dalam detik), sedangkan sumbu vertikal merepresentasikan amplitudo. Sinyal original memiliki amplitudo yang lebih tinggi dan fluktuasi yang lebih kompleks, sedangkan noise tampak lebih kecil dengan amplitudo yang rendah dan perubahan yang lebih cepat. Ketika kedua sinyal ini digabungkan, noise "menempel" pada sinyal original, menyebabkan amplitudo gabungan meningkat dan gelombang menjadi lebih padat.

Grafik kedua (bagian bawah) menunjukkan sinyal gabungan secara keseluruhan, yang ditampilkan dalam warna ungu. Sinyal ini merupakan hasil penjumlahan dari sinyal original dan noise. Terlihat bahwa amplitudo sinyal gabungan lebih besar dibandingkan sinyal original, mencerminkan penambahan noise ke dalam audio asli. Fluktuasi pada sinyal gabungan menjadi lebih kompleks karena adanya komponen frekuensi noise yang bercampur dengan sinyal asli. Selain itu, perubahan amplitudo pada sinyal gabungan mengikuti pola dominan dari sinyal original, tetapi dengan gangguan yang muncul akibat noise berfrekuensi tinggi. Proses ini menunjukkan bagaimana noise dapat mengganggu kualitas audio asli. Dengan informasi ini, langkah selanjutnya dalam proyek adalah menerapkan filter noise reduction, seperti low-pass filter dengan Finite Impulse Response (FIR), untuk menghilangkan komponen noise tersebut dan mengembalikan kualitas audio mendekati sinyal original.

5. Implementasi Filtering untuk mereduksi Noise

Proses yang dilakukan mencakup proses reduksi noise, penerapan filter low-pass, dan amplifikasi sinyal untuk meningkatkan kualitas audio. Pertama, sinyal audio yang sudah mengandung noise diolah dengan metode *noise reduction* menggunakan pustaka *noisereduce*, di mana segmen awal audio digunakan sebagai *noise profile*. Selanjutnya, sinyal hasil reduksi noise diberikan filter low-pass berbasis *Finite Impulse Response (FIR)* untuk meredam frekuensi tinggi yang tidak diinginkan, dengan cutoff 200 Hz. Setelah proses filtering, dilakukan amplifikasi sinyal dengan perhitungan Root Mean Square (RMS) untuk menyesuaikan tingkat gain.

Output dari kode filtering ini menunjukkan proses peningkatan kualitas audio melalui tiga tahapan utama yang divisualisasikan dalam tiga grafik.



Grafik Pertama - "Gelombang Sinyal Original Audio dan Noise Audio": Grafik ini menampilkan audio gabungan yang berisi sinyal asli (original) dengan noise tambahan. Sinyal ditampilkan dalam warna ungu, dan terlihat amplitudo yang bervariasi secara kompleks karena adanya gangguan noise. Grafik ini merepresentasikan audio awal sebelum dilakukan pemrosesan.

Grafik Kedua - "Enhanced Audio with Noise Reduction and Low-Pass Filter": Pada tahap ini, noise reduction diterapkan menggunakan library `noisereduce` untuk mengurangi komponen noise. Selanjutnya, FIR low-pass filter diterapkan untuk memblokir frekuensi tinggi di atas 200 Hz, yang umumnya mendominasi noise. Hasilnya adalah audio dengan amplitudo yang lebih bersih, di mana komponen noise telah berkurang secara signifikan, sehingga sinyal asli lebih terlihat jelas. Namun, amplitudo masih dalam skala yang sama dengan audio awal.

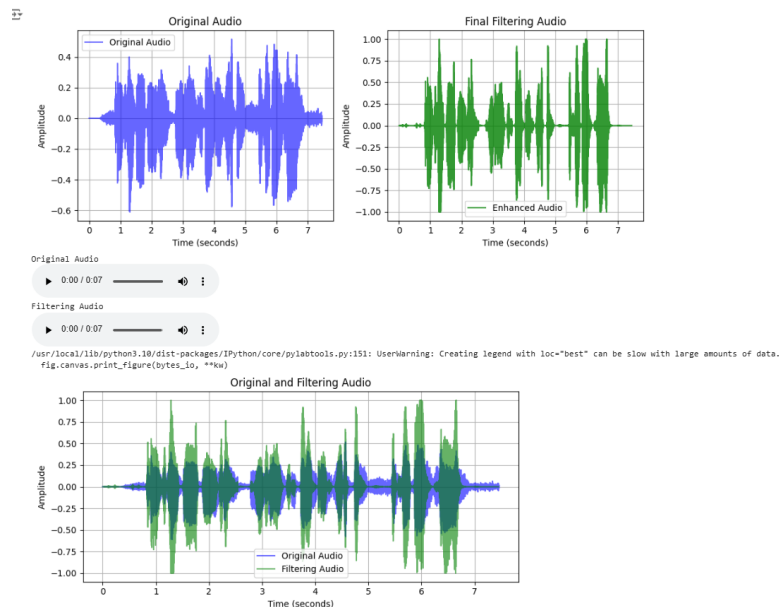
Grafik Ketiga - "Final Enhanced Audio (Noise Reduced, Filtered & Louder)": Tahap terakhir adalah penyesuaian amplitudo untuk membuat audio lebih nyaring. RMS (Root Mean Square) dari sinyal dihitung dan diperkuat dengan faktor tertentu agar amplitudo meningkat secara konsisten. Grafik ini memperlihatkan sinyal audio yang telah

melalui semua proses—noise reduction, filtering, dan amplifikasi. Hasil akhirnya memiliki amplitudo yang lebih besar, sinyal lebih bersih, dan noise sudah berkurang drastis.

Dengan visualisasi ini, dapat disimpulkan bahwa proses filtering berhasil menghilangkan noise frekuensi tinggi, memperbaiki kualitas sinyal, dan membuat audio lebih nyaman didengar. Audio akhir dapat diputar untuk mendengar perbedaan signifikan dibandingkan audio gabungan awal.

6. Perbandingan Gelombang Sinyal Original Audio dan Hasil Filtering

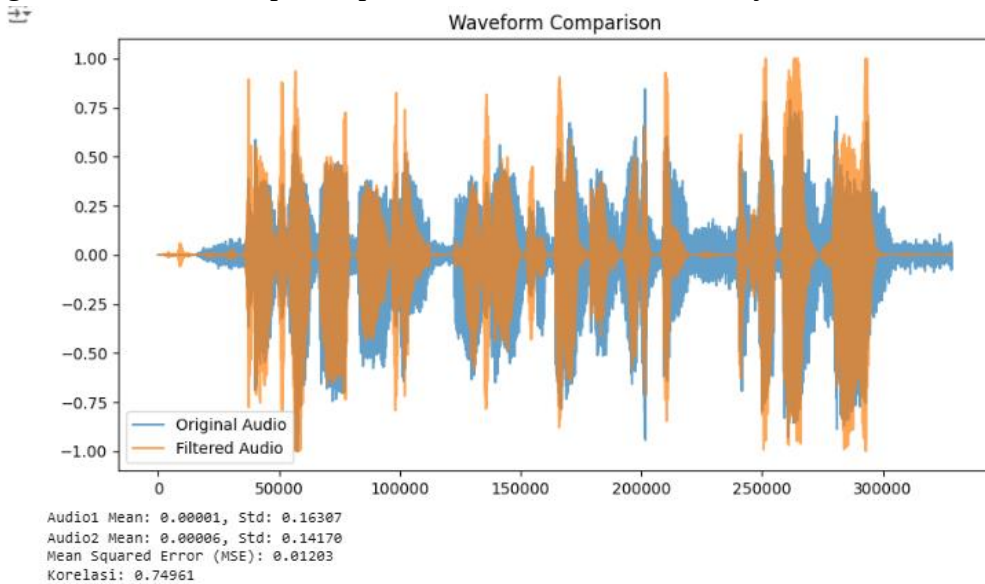
Setelah diimplementasikannya, maka selanjutnya dilakukannya perbandingan berdasarkan visualisasi gelombang sinyal original audio dengan audio gabungan original dan noise yang telah melalui proses penerapan filter low-pass, dan amplifikasi sinyal sebelumnya



Berdasarkan Visualisasi pertama menampilkan dua subplot: grafik kiri menunjukkan audio original berwarna biru yang nantinya akan dijadikan perbandingan, selanjutnya pada grafik kanan adalah original audio dengan audio yang telah digabung dengan noise lalu telah melewati proses reduksi noise, low-pass filtering, dan amplifikasi, sehingga sinyal menjadi lebih bersih dan stabil. Selanjutnya, kedua audio diputar untuk memberikan perbandingan auditori, di mana audio hasil filtering terdengar lebih jernih dan nyaman didengar. Grafik terakhir meng-overlay kedua sinyal dalam satu plot, dengan warna biru mewakili audio original dan warna hijau untuk audio hasil filtering. Perbedaan signifikan terlihat jelas, di mana sinyal hasil filtering memiliki noise yang berkurang drastis dan amplitudo yang lebih konsisten dibandingkan audio asli. Secara keseluruhan, output ini menunjukkan keberhasilan proses filtering dalam meningkatkan kualitas audio dengan mengurangi noise dan mempertajam sinyal utama.

7. Perbandingan Korelasi Original Audio dengan Audio Hasil Filtering

Pada proses terakhir yaitu, dilakukannya Perbandingan Korelasi Original Audio dengan Audio Hasil Filtering. Hasil perbandingan antara original audio dan audio hasil filtering menunjukkan adanya peningkatan kualitas audio dengan pengurangan noise yang signifikan, namun tetap mempertahankan struktur utama sinyal.



Grafik waveform comparison memperlihatkan gelombang audio asli (berwarna biru) dan hasil filtering (berwarna oranye) yang memiliki pola serupa secara keseluruhan, meskipun amplitudo hasil filtering tampak lebih stabil dan lebih bersih. Dari segi statistik, nilai mean dari original audio adalah 0.00031, sementara filtered audio memiliki nilai 0.00085, yang menunjukkan bahwa kedua audio berosilasi secara seimbang di sekitar titik nol. Sementara itu, standard deviation (std) untuk original audio adalah 0.16307, dan untuk filtered audio lebih rendah, yaitu 0.14170, menandakan bahwa variasi amplitudo berkurang setelah filtering.

Selain itu, Mean Squared Error (MSE) sebesar 0.01203 menunjukkan bahwa perbedaan antara kedua sinyal cukup kecil, yang berarti proses filtering berhasil mempertahankan sebagian besar informasi penting dari sinyal asli. Nilai korelasi sebesar 0.74961 juga mengindikasikan adanya hubungan yang cukup kuat antara original audio dan hasil filtering, menegaskan bahwa proses filtering berhasil mempertahankan struktur utama gelombang suara. Secara keseluruhan, hasil ini menunjukkan bahwa audio hasil filtering memiliki noise yang lebih rendah, amplitudo yang lebih stabil, dan kualitas yang lebih bersih dibandingkan audio asli, dengan perubahan minimal terhadap struktur sinyal aslinya.

8. Graphic User Interface (GUI) Cleanify Pro

Cleanify Pro adalah sebuah aplikasi berbasis GUI (*Graphical User Interface*) yang dirancang untuk meningkatkan kualitas audio dengan menerapkan teknik filtering menggunakan Finite Impulse Response (FIR) filter. Aplikasi ini berfungsi untuk mengurangi gangguan atau noise pada file audio, sehingga menghasilkan audio yang lebih jernih dan nyaman didengarkan.

Fitur Utama Cleanify Pro:

1. Upload File Audio

Pengguna dapat dengan mudah mengunggah file audio dalam format seperti .wav untuk diproses.

2. Real-Time Feedback

Selama proses filtering, pengguna akan mendapatkan notifikasi real-time mengenai status dan hasil proses audio mereka.

3. Proses Filtering dengan FIR

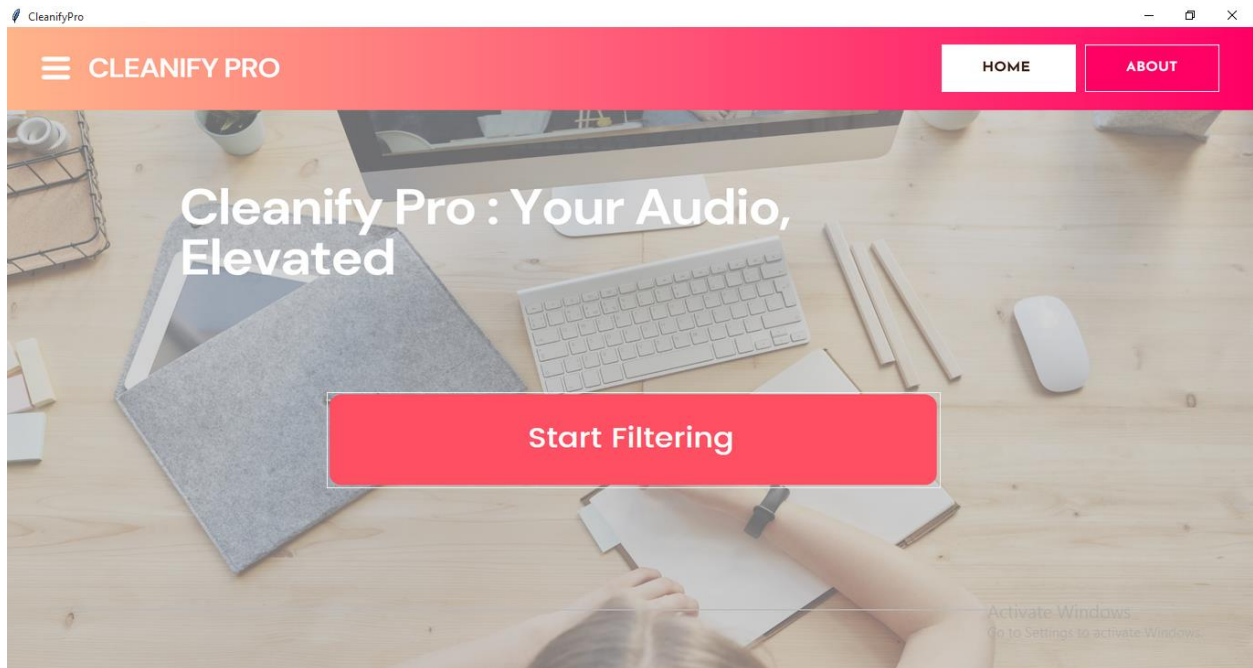
Cleanify Pro menggunakan pendekatan FIR filter, sebuah teknik filtering digital yang unggul dalam menghasilkan respons filter yang stabil tanpa distorsi fase.

4. Pengunduhan Hasil Audio

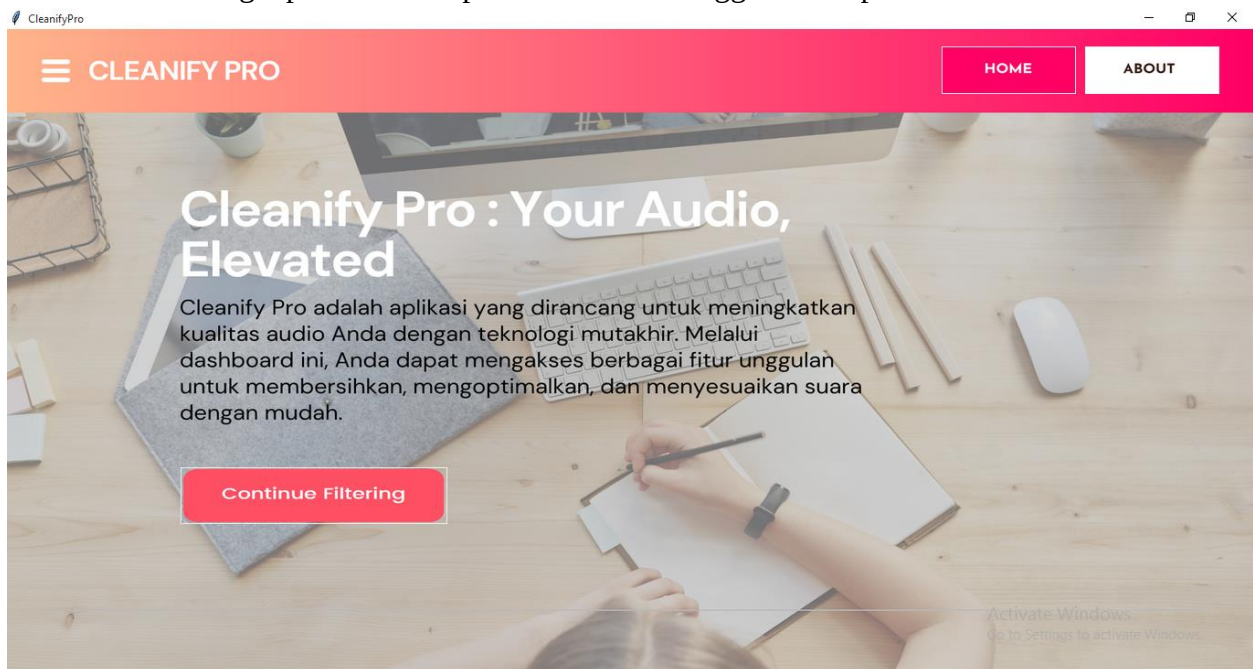
Setelah proses filtering selesai, pengguna dapat mengunduh file audio yang telah ditingkatkan kualitasnya dalam format .wav.

Manfaat Cleanify Pro:

1. Menghilangkan noise yang tidak diinginkan pada rekaman suara.
2. Mempertahankan kualitas audio dengan distorsi minimal.
3. Cocok untuk berbagai aplikasi, seperti pengolahan musik, podcast, atau rekaman suara profesional.

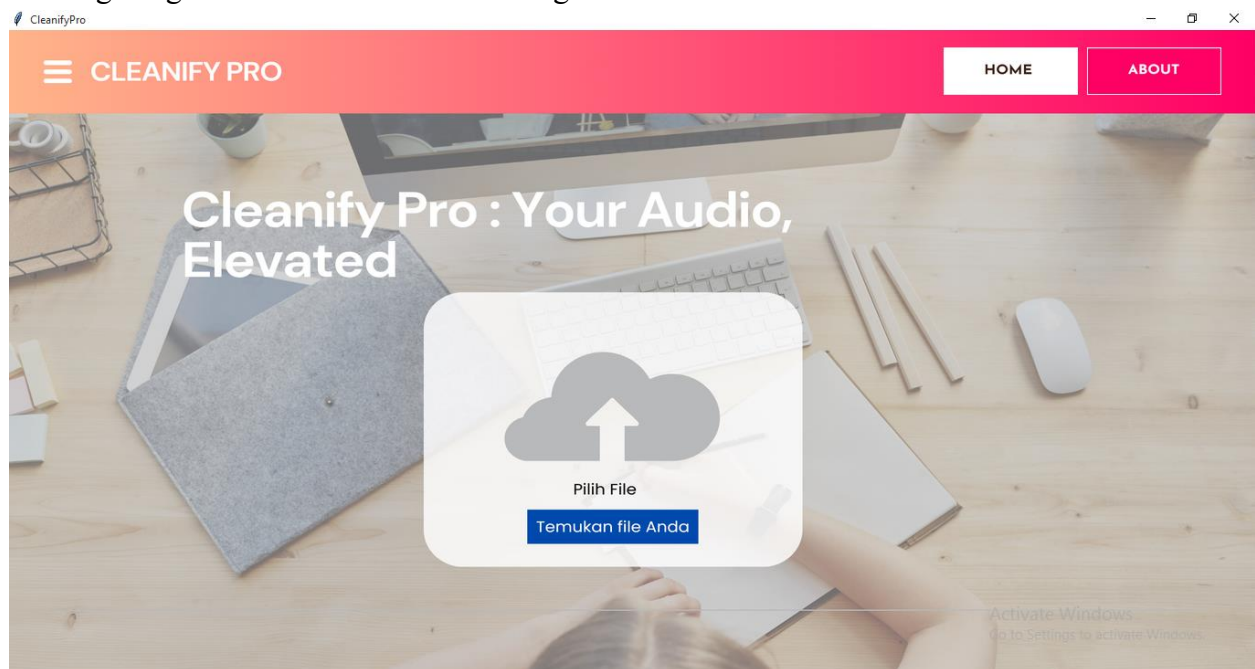


Fitur "Home" pada Cleanify Pro berfungsi sebagai halaman utama aplikasi yang memberikan tampilan untuk memulai proses penyaringan audio. Pada halaman ini, pengguna dapat melihat slogan "Your Audio, Elevated," yang memberikan tujuan aplikasi untuk meningkatkan kualitas audio. Terdapat tombol "Start Filtering" yang menjadi akses utama untuk memulai fitur filtering. Desain ini mempermudah pengguna dalam bernavigasi tanpa kebingungan, menjadikan fitur "Home" sebagai pusat kontrol pertama dalam menggunakan aplikasi.

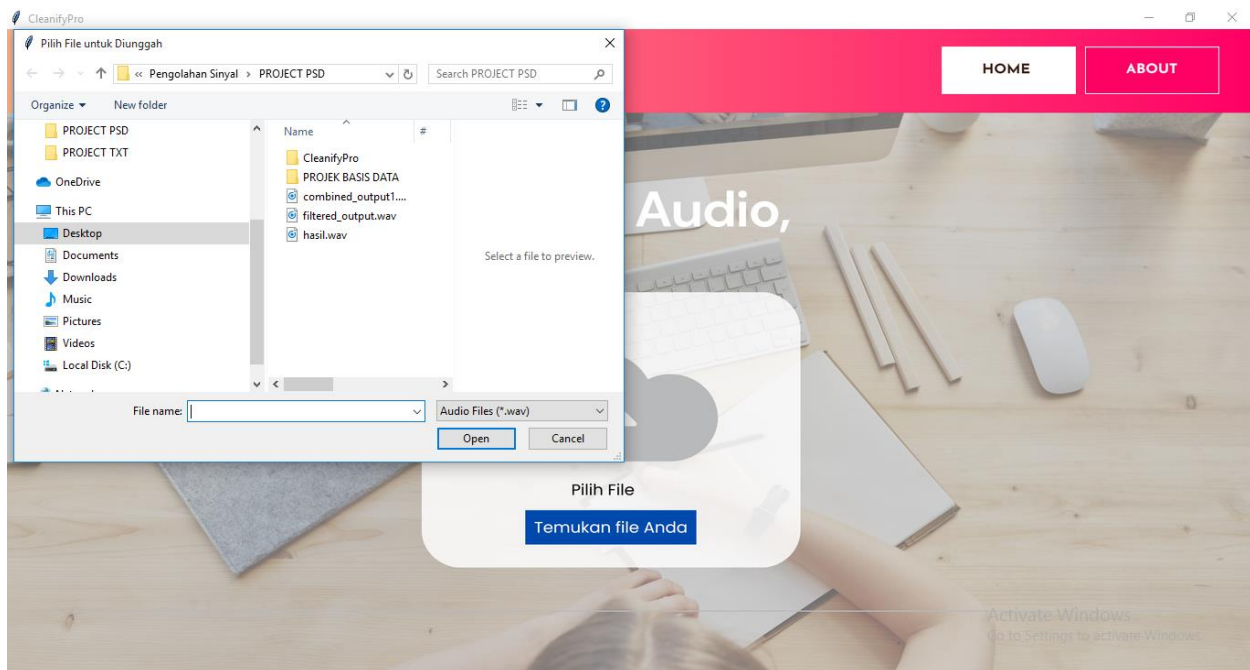


Setelah user diarahkan ke halaman utama, user juga dapat mengakses tombol "About" yang terletak di pojok kanan atas. Tombol ini berisi informasi lengkap mengenai Cleanify Pro, sehingga pengguna dapat memahami lebih mendalam tentang platform ini. Selain itu, pada halaman

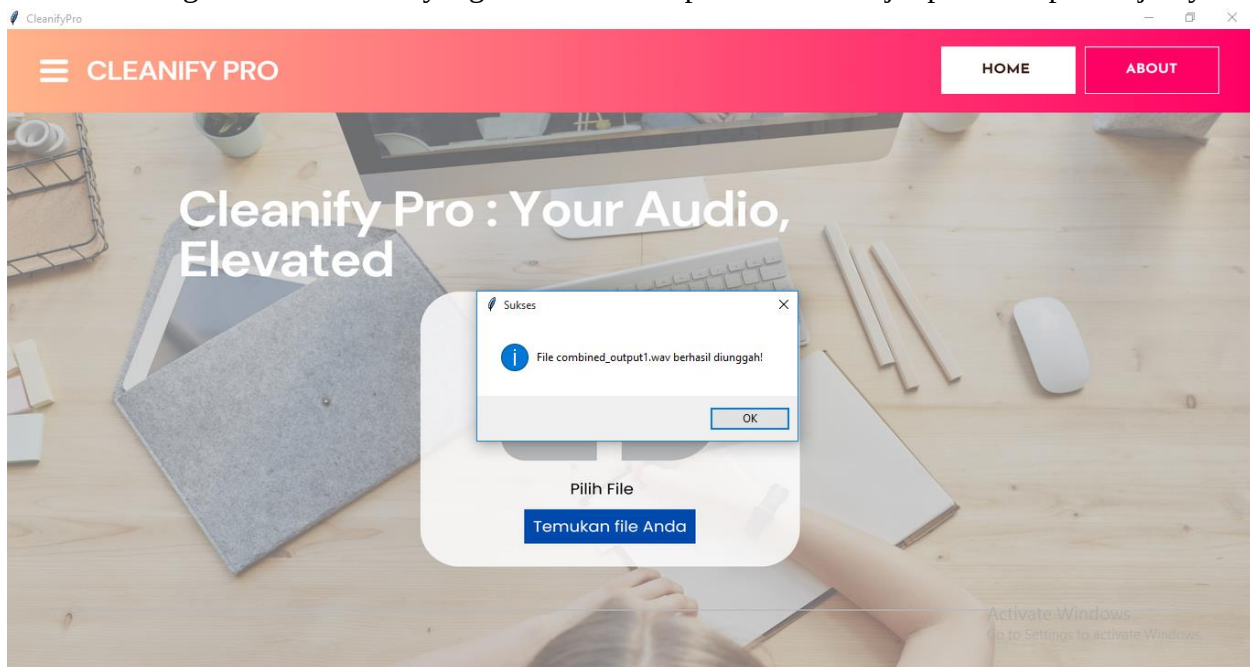
tersebut, user juga memiliki opsi untuk melakukan proses filtering melalui tombol “Continue Filtering” yang selanjutnya, user akan diarahkan langsung pada home page dan memulai proses filtering dengan klik tombol “Start Filtering” .



Setelah user menyetujui untuk memulai proses penyaringan dengan mengklik tombol “Start Filtering” seperti yang telah ditampilkan sebelumnya, user akan diarahkan ke langkah berikutnya. Pada tahap ini, user diminta untuk memilih file yang akan diproses menggunakan Cleanify Pro dengan mengklik tombol “Temukan File Anda”. Fitur ini dirancang untuk memudahkan user dalam mengakses direktori file mereka, sehingga proses pemilihan file yang sesuai dapat dilakukan dengan cepat dan efisien sebelum melanjutkan ke tahap filtering.

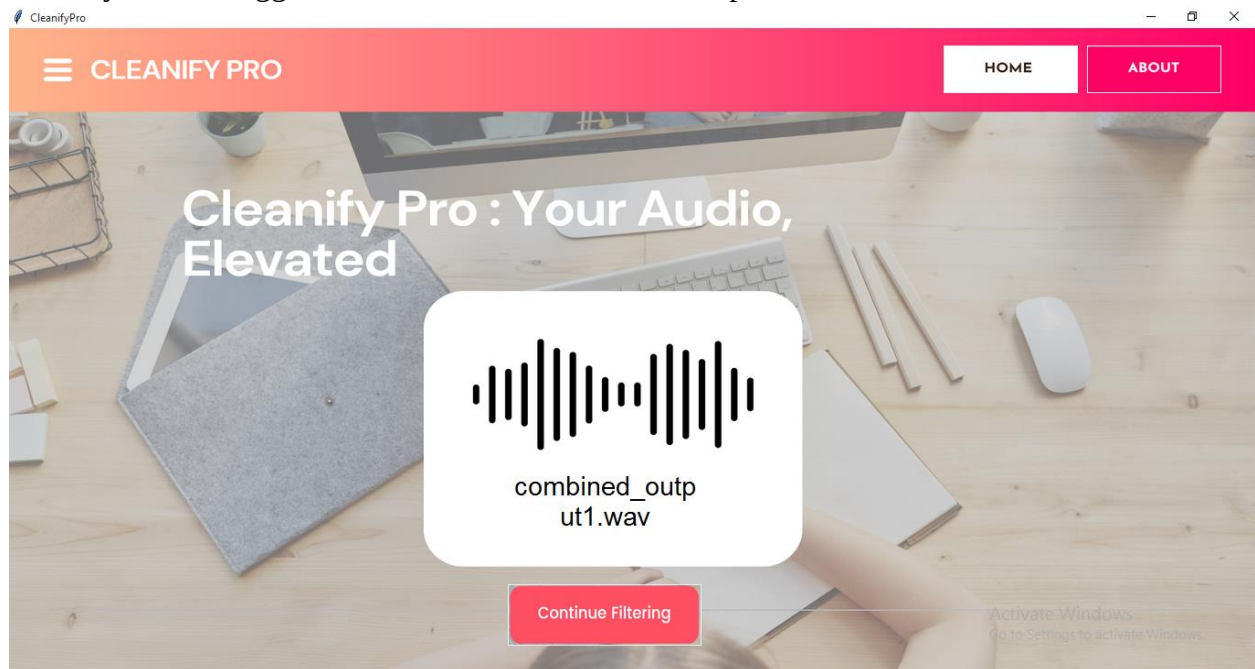


Apabila user telah mengklik tombol “Temukan File Anda”, maka langkah selanjutnya adalah user akan diarahkan ke file direktori pada perangkat user. Pada tahap ini, user dapat memilih file audio dengan format .WAV yang sesuai untuk diproses lebih lanjut pada tahap selanjutnya.

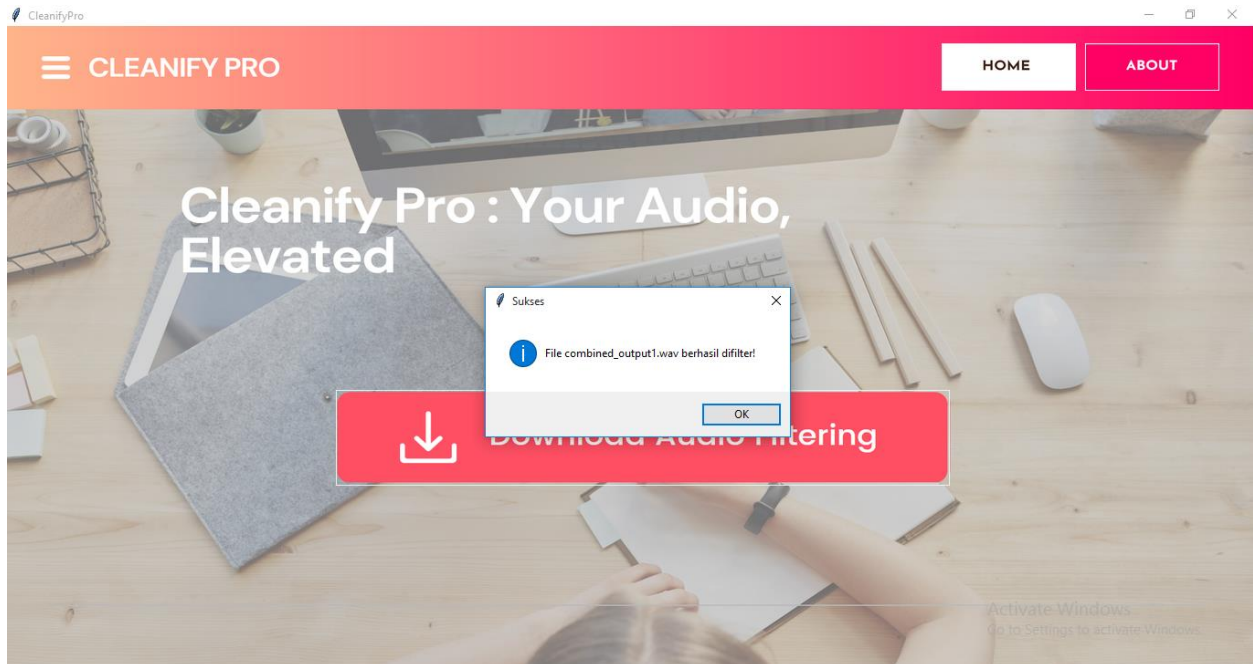


Apabila user telah menemukan file dengan format yang sesuai untuk dilakukan proses filtering, maka langkah berikutnya adalah user akan menerima notifikasi berupa “message box”. Notifikasi ini memberikan informasi bahwa file yang dipilih telah berhasil diunggah. Selanjutnya, apabila user mengklik tombol “Ok”, hal ini menunjukkan persetujuan user untuk melanjutkan proses pada file audio yang telah dipilih. Selain itu, fitur ini juga memastikan bahwa user telah

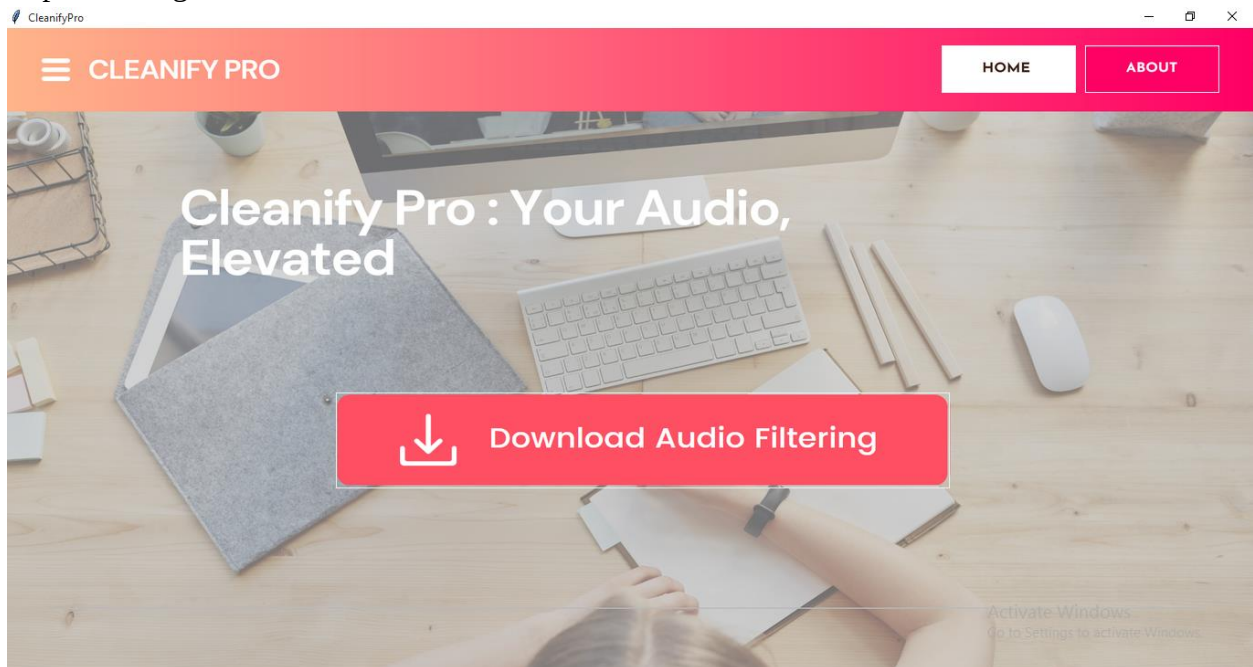
memeriksa kembali file yang akan diproses lebih lanjut melalui tahap Filtering menggunakan Cleanify Pro, sehingga meminimalkan kesalahan dalam pemilihan file.



Setelah pengguna menerima notifikasi bahwa file audio telah berhasil diunggah, langkah berikutnya adalah mengklik tombol “Continue Filtering”. Tombol ini berfungsi sebagai konfirmasi akhir dari pengguna untuk memulai proses *filtering* pada file yang telah dipilih. Dengan menekan tombol ini, pengguna diarahkan ke tahap pemrosesan lebih lanjut, di mana aplikasi *Cleanify Pro* akan menerapkan algoritma atau teknik tertentu untuk meningkatkan kualitas audio sesuai dengan kebutuhan pengguna. Tombol ini dirancang agar mudah diakses dan memberikan pengalaman pengguna yang intuitif, memastikan bahwa pengguna memiliki kontrol penuh atas file yang sedang diproses. Langkah ini juga menegaskan komitmen aplikasi untuk memberikan hasil yang optimal dengan meminimalkan risiko kesalahan dalam pemilihan dan pemrosesan file.

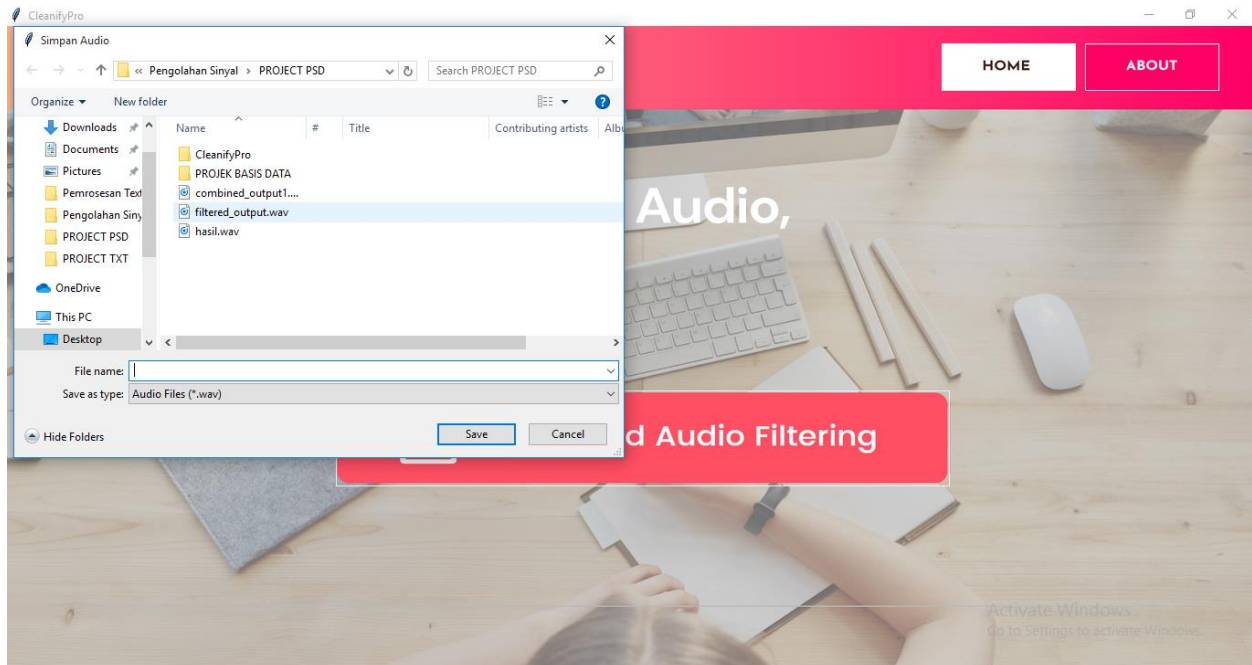


Setelah user mengklik tombol “Continue Filtering” pada tahap sebelumnya, langkah selanjutnya adalah user akan menerima notifikasi berupa “message box”. Notifikasi ini berfungsi untuk mengonfirmasi bahwa file yang telah diunggah sebelumnya telah berhasil diproses melalui tahap penyaringan menggunakan Cleanify Pro. Pesan ini dirancang untuk memberikan kepastian kepada user bahwa proses filtering telah selesai dengan sukses, sehingga file audio yang dihasilkan siap untuk digunakan atau diunduh sesuai kebutuhan.

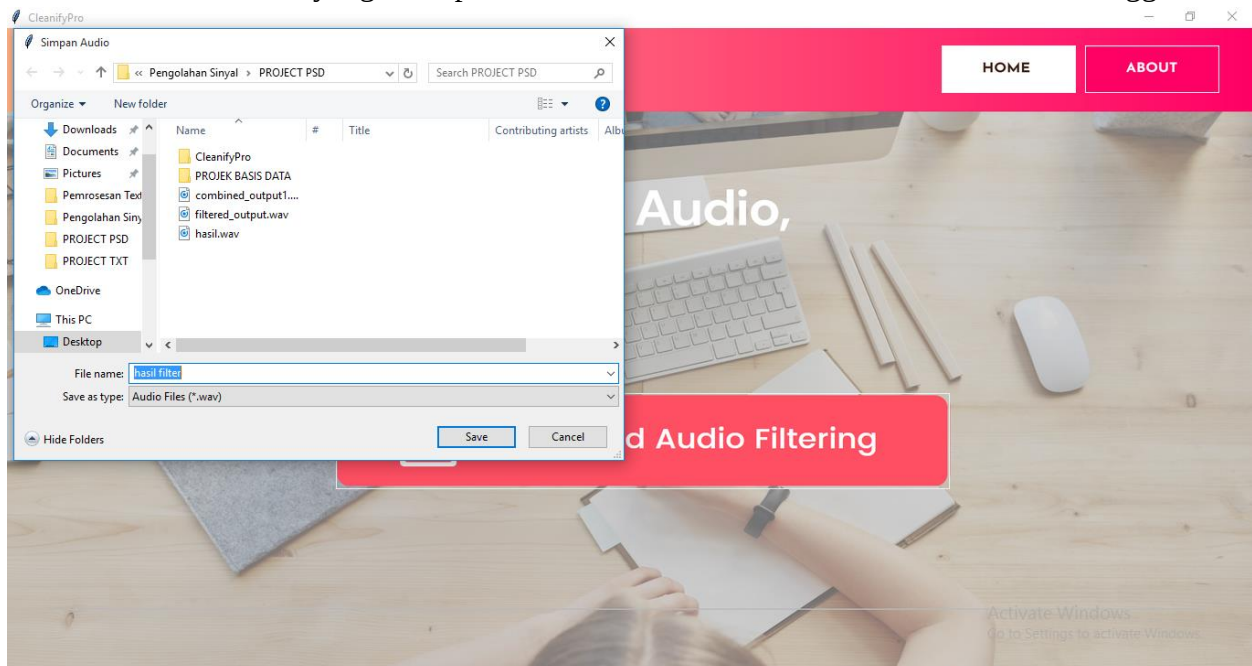


Fitur Download Audio Filtering pada aplikasi Cleanify Pro berfungsi untuk mengunduh hasil penyaringan audio yang telah diproses. Ketika tombol ini ditekan, aplikasi akan menyimpan

file audio yang telah melewati proses penyaringan menggunakan filter FIR (Finite Impulse Response).

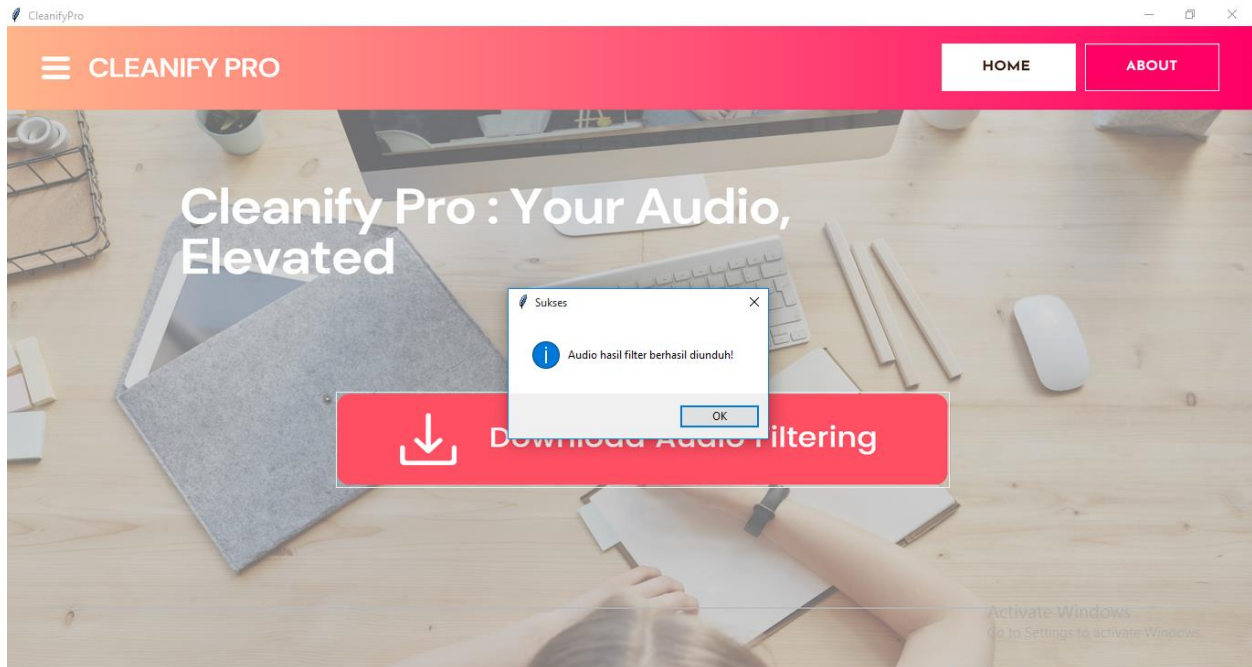


Setelah pengguna menekan tombol Download Audio Filtering, dialog penyimpanan akan muncul, memberikan opsi untuk memilih folder penyimpanan, menentukan nama file, dan memastikan format file yang disimpan adalah .wav, sesuai standar audio berkualitas tinggi.



File hasil pemrosesan audio yang telah difilter dapat disimpan Misalkan dengan nama filter.wav. Proses ini dilakukan melalui dialog penyimpanan file, di mana pengguna dapat menentukan nama file dan lokasi penyimpanannya. Dalam hal ini, file disimpan dengan format .wav. Sebagai contoh, jika file disimpan di direktori Pengolahan Sinyal > PROJECT PSD, maka

file tersebut akan tersimpan dengan nama filter.wav dan dapat digunakan untuk analisis lebih lanjut, pemutaran, atau dokumentasi hasil proses filtering.



Setelah pengguna menekan tombol Save untuk menyimpan file audio, aplikasi akan menampilkan notifikasi "Audio hasil filter.wav berhasil diunduh." yang mengonfirmasi bahwa file filter.wav berhasil diunduh. Notifikasi ini memberitahu pengguna bahwa file yang telah diproses dan disaring telah berhasil disimpan di perangkat mereka, dengan nama filter.wav, memberikan informasi bahwa proses penyimpanan telah selesai.

 hasil filter.wav	27/12/2024 23:42	WAV File	2.795 KB
---	------------------	----------	----------

File hasil pengunduhan dengan nama 'hasil filter.wav' akan secara otomatis tersimpan di lokasi yang telah ditentukan pada proses sebelumnya. File audio tersebut telah melalui tahap penyaringan menggunakan FIR Filter, sehingga menghasilkan *audio clean* yang telah dibersihkan dari audio noise.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Kesimpulan dari penelitian ini menunjukkan bahwa penerapan teknologi pengolahan sinyal digital berbasis Finite Impulse Response (FIR) menggunakan low-pass filter efektif dalam mengurangi noise pada sinyal audio. Penggunaan filter ini berhasil meningkatkan kejernihan dan kualitas audio, sehingga suara yang dihasilkan lebih jelas dan nyaman didengar. Hasil evaluasi juga menunjukkan perbedaan signifikan antara sinyal audio sebelum dan setelah pengolahan, dengan pengukuran korelasi dan Mean Squared Error (MSE) yang mendukung efektivitas metode yang digunakan

5.2 Saran

Pengembangan lebih lanjut, disarankan untuk melakukan uji coba pada berbagai jenis sinyal audio, termasuk musik dan suara vokal, serta mengembangkan perangkat lunak yang dapat mengimplementasikan teknologi ini secara real-time. Penelitian lebih lanjut juga dapat mengeksplorasi metode lain dalam pengolahan sinyal, termasuk kombinasi antara FIR dan IIR filter, untuk analisis noise yang lebih kompleks. Dengan demikian, penelitian ini diharapkan dapat memberikan kontribusi yang signifikan dalam meningkatkan pengalaman pengguna dalam mendengarkan audio berkualitas tinggi

DAFTAR PUSTAKA

- Gunawan.Dadang, Juwono,F.T. (2012). *PENGOIAHAN SINYAL DIGITAL DENGAN PEMROGRAMAN MATHLAB (EDISI PERTAMA)*. Graha Ilmu
- Maulana Yusup, Rian. (2023). *Pemrosesan Sinyal Digital*. ResearchGet.Net.
- Salih, A. O. M. (2017). *Audio noise reduction using Low pass filters*. Open Access Library Journal, 4(11), 1-7.
- Santoso, T. B., Octavianto, H., & Dutono, T. (n.d.). *Implementasi Filter IIR secara Real Time pada TMS 32C5402*. Laboratorium Sinyal, Politeknik Elektronika Negeri Surabaya .
- .